

Instructor solutions: session 02

Module 1 · 9 September 2026 · Roman Jurowetzki

Every exercise from both student notebooks, with the solution, what students actually hand in, and something to say when they are stuck. The numbers come from the frozen snapshot, so you can check someone's screen without rerunning anything.

The solution code here is copied straight out of the student notebooks. If you change a solution there, rebuild this file with `courses/ds4b-m1-3-pandas-2026/build_instructor_notebook.py` instead of editing it by hand.

This one is for you, not for Moodle. Timing, run of show, scaffolding and the Colab notes live in [instructor-guide.md](#). This is the half that runs.

Running it

1. Run the helper cell below, once.
2. Run the Part 1 setup cell, then anything in Part 1.
3. Run the Part 2 setup cell before anything in Part 2.

Both setup cells replay the matching student notebook with its output suppressed, so everything the solutions need is in scope and identical to what students have on screen.

Watch out: Part 2's setup rebinds `df` to the Spotify data. If you jump back to a Part 1 solution afterwards, rerun the Part 1 setup first.

```

# Run once. Replays a student notebook quietly so its objects exist here too.
import json
from pathlib import Path
from urllib.request import urlopen

import matplotlib.pyplot as plt
from IPython.utils.capture import capture_output

RAW_NOTEBOOKS = "https://raw.githubusercontent.com/aaubs/ds-master/codex/m1-pandas-2026/notebooks"

def run_student_notebook(filename):
    """Execute every code cell of a student notebook in this kernel, silently."""
    candidates = [Path(filename),
                  Path("notebooks") / filename,
                  Path("..") / "notebooks" / filename,
                  Path("ds-master/notebooks") / filename]

    source = None
    for path in candidates:
        if path.exists():
            source = path.read_text()
            print(f"Replaying {path}")
            break

    if source is None:
        url = f"{RAW_NOTEBOOKS}/{filename}"
        source = urlopen(url).read().decode()
        print(f"Replaying {url}")

    with capture_output():
        for cell in json.loads(source)["cells"]:
            if cell["cell_type"] == "code":
                exec("".join(cell["source"]), globals())

    plt.close("all")
    print("Done. Every object from that notebook is now in scope.")

```

Part 1 · From control flow to pandas

```

run_student_notebook("M1_01_control_flow_to_pandas.ipynb")
print("Catalog shape:", df.shape)

```

```

Replaying notebooks/M1_01_control_flow_to_pandas.ipynb
Done. Every object from that notebook is now in scope.
Catalog shape: (2959, 13)

```

Practice 1 · Collect the free-course titles (loop and `if`)

Write a loop that collects the titles of free courses into `free_titles`.

Answer: `['Python Basics Lab', 'Security Lab']`, in that order. The proxy loop above it prints 3,550 if untouched.

What you will see	What went wrong	Say this
A list of dictionaries	They appended <code>course</code> , not <code>course["course_title"]</code>	"Print one element. Is that a title, or a whole record?"
<code>[]</code>	<code>course["price"] == "0"</code> , comparing a number to text	"What type is <code>price</code> ? Try <code>type(course['price'])</code> ."
One title only	<code>free_titles = []</code> ended up inside the loop	"Where does the empty list need to exist before the loop starts?"
<code>AttributeError: 'NoneType'</code>	They kept the starter's <code>= None</code> and called <code>.append</code>	" <code>None</code> is not a list yet."

Keep this loop on the board. You will point back at it twice: once for the boolean mask in section 2, once for the `.loc` filter in section 3.

```
free_titles = []
for course in sample_courses:
    if course["price"] == 0:
        free_titles.append(course["course_title"])
print(free_titles)
```

```
['Python Basics Lab', 'Security Lab']
```

Practice 2 · Combine two array conditions

Use `&` to find Web Development courses that are popular.

Answer: `[ True False True False False]`

Worth staging live: `&` binds tighter than `==`, so dropping the parentheses does not give a wrong answer, it raises. Running the next cell beats explaining operator precedence in the abstract.

Students who write `and` instead of `&` get a different error, about the truth value of an array being ambiguous. Both errors mean the same thing. `and` asks one yes-or-no question, `&` asks it once per element.

```
web_and_popular = (subjects == "Web Development") & (enrollments >= 1000)
print(web_and_popular)
```

```
[ True False  True False False]
```

```
# Live demo: the two mistakes, and what each one says.
for label, attempt in [
    ("no parentheses", lambda: subjects == "Web Development" & enrollments >= 1000),
    ("and instead of &", lambda: (subjects == "Web Development") and (enrollments >= 1000)),
]:
    try:
        attempt()
        print(f"{label}: no error (unexpected)")
    except Exception as exc:
        print(f"{label}: {type(exc).__name__}: {exc}")
```

no parentheses: `TypeError: ufunc 'bitwise_and' not supported for the input types, and the inputs could not be safely coerced to any supported types according to the casting rule 'safe'`

and instead of &: `ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`

Practice 3 · A paid, popular mask on the real catalog

Find courses that are paid and have at least 100,000 subscribers, then show three columns with `.loc`.

Answer: 2 rows. Warn them, because two rows looks like a bug to someone expecting a table. It is correct for this file.

Things to watch:

`df["is_paid"] == True` works, but the column is already True/False. Ask what the comparison adds. The hint in the student notebook now says this.

`df[df["is_paid"]][["course_title", ...]]` also works, and it is the habit that produces `SettingWithCopyWarning` the first time they try to assign. Steer to `.loc[mask, columns]` now, while it costs nothing.

`and` instead of `&`, again. Second sighting, so let a student explain it this time.

```
# is_paid already holds True/False, so it goes straight into the &.
paid_and_popular = df["is_paid"] & (df["num_subscribers"] >= 100000)
display(df.loc[paid_and_popular, ["course_title", "price", "num_subscribers"]].head())
```

	course_title	price	num_subscribers
2587	The Web Developer Bootcamp	200	121584
2589	The Complete Web Developer Course 2.0	200	114512

Practice 4 · One constrained question

Which subject has the highest average subscribers among `Beginner Level` courses?

Answer: Web Development, about 7,864 mean subscribers, ahead of Business Finance (2,331), Musical Instruments (1,627) and Graphic Design (1,310).

Things to watch:

No filter, so they group the whole catalog and get the section 4 numbers instead. Ask which rows the answer is based on.

`.sum()` instead of `.mean()`. The question said average. This is what section 4's warning was for, so make them say why the totals mislead. The demo cell below shows the ranking moving.

`level` spelled from memory. `"Beginner"` returns an empty frame. `df["level"].unique()` is the rescue: the four values are `Intermediate Level`, `All Levels`, `Beginner Level`, `Expert Level`.

Reporting `.max()`, which is the number rather than the subject. `.index[0]` after sorting, or `.idxmax()`, gives the label.

```
beginner = df.loc[df["level"] == "Beginner Level"]
average_by_subject = (
    beginner.groupby("subject")["num_subscribers"]
        .mean()
        .sort_values(ascending=False)
)
print(average_by_subject)
print("Top subject:", average_by_subject.index[0])
```

```
subject
Web Development      7863.604651
Business Finance     2330.957529
Musical Instruments  1627.000000
Graphic Design       1310.152941
Name: num_subscribers, dtype: float64
Top subject: Web Development
```

```
# Live demo for the totals-versus-averages point: the ranking moves.
by_subject = df.groupby("subject")["num_subscribers"]
comparison = pd.DataFrame({
    "courses": df.groupby("subject").size(),
    "total_subscribers": by_subject.sum(),
    "average_subscribers": by_subject.mean().round(0),
})
print("Ranked by total:")
display(comparison.sort_values("total_subscribers", ascending=False))
print("Ranked by average:")
display(comparison.sort_values("average_subscribers", ascending=False))
```

Ranked by total:

	courses	total_subscribers	average_subscribers
subject			
Web Development	976	7301548	7481.0
Business Finance	968	1738412	1796.0
Graphic Design	447	907807	2031.0
Musical Instruments	568	779127	1372.0

Ranked by average:

	courses	total_subscribers	average_subscribers
subject			
Web Development	976	7301548	7481.0
Graphic Design	447	907807	2031.0
Business Finance	968	1738412	1796.0
Musical Instruments	568	779127	1372.0

Business Finance is second on totals and third on averages, because it carries 968 courses to Graphic Design's 447. Get that sentence out of a student before the break: a bigger total can just mean a bigger catalog.

Optional extension · Price bands with `pd.cut`

Only if the room is ahead. `pd.cut` is the `price_tier` ladder from section 1.2 as one call, which makes a satisfying callback.

Two things trip people up. Bin edges are closed on the right, so free courses need a lower edge below zero (`-1`) to land in the first band. And the edges have to increase, so `bins=[0, 0, 100, inf]` raises. If someone asks about conditions that are not ranges, price and subject say, that is `np.select` , and naming it without teaching it today is fine.

```
# pd.cut turns a number into a named band. The bins are the cut points and
# each band is closed on the right: (-1, 0] is Free, (0, 100] is Standard, the rest
# Premium.
# This is the price_tier function from section 1.2, written as one pandas call.
df["price_tier"] = pd.cut(
    df["price"],
    bins=[-1, 0, 100, float("inf")],
    labels=["Free", "Standard", "Premium"],
)
display(df[["course_title", "price", "price_tier"]].head())
```

	course_title	price	price_tier
0	Financial Modeling for Business Analysts and C...	45	Standard
1	How To Maximize Your Profits Trading Options	200	Premium
2	Trading Penny Stocks: A Guide for All Levels I...	150	Premium
3	Investing And Trading For Beginners: Mastering...	65	Standard
4	Trading Stock Chart Patterns For Immediate Exp...	95	Standard

Part 2 · Pandas deep dive

Run the setup cell before any Part 2 solution. It takes a few seconds, since it replays the whole student notebook, charts included, with the output suppressed.

```
run_student_notebook("M1_02_pandas_deep_dive.ipynb")
print("Membership rows:", len(membership))
print("Track rows:", len(tracks))
print("Joined rows:", len(joined))
```

Replaying notebooks/M1_02_pandas_deep_dive.ipynb
 Done. Every object from that notebook is now in scope.
 Membership rows: 32510
 Track rows: 28356
 Joined rows: 32510

Exercise 1 · What is actually missing? (4 min)

Answer: `title` and `artist` have 5 missing each. `energy` and `danceability` have none.

The decision matters more than the count. The follow-up question, would you drop every row with a missing field, is the real one. No: the rows with an unknown artist still carry observed audio features, and the comparison is about energy by genre. Dropping them throws away usable measurements to tidy a column nobody is analysing.

Things to watch:

The `.dropna()` reflex, applied to the whole frame before looking at anything.

`df.isna().sum().sum()`, one grand total, which answers nothing.

Treating `"Unknown artist"` as a finding. It is a display label and recovers no identity. If a student later groups by `artist_display`, that is where it bites.

```
# Solution 1: missingness in the fields used for interpretation.
exercise_missing = df[["title", "artist", "energy", "danceability"]].isna().sum()
display(exercise_missing)
display(df.loc[df["artist"].isna(), ["track_id", "artist", "genre", "energy"]])
```

```
title          5
artist         5
energy         0
danceability    0
dtype: int64
```

	track_id	artist	genre	energy
8151	69gRFGOWY9OMpFJgFol1u0	NaN	rap	0.821
9282	5cjecvX0CmC9gK0Laf5EMQ	NaN	rap	0.659
9283	5TTzhRSWQS4Yu8xTgAuq6D	NaN	rap	0.820
19568	3VKFip3OdAvv4OfNTgFWeQ	NaN	latin	0.919
19811	69gRFGOWY9OMpFJgFol1u0	NaN	latin	0.821

Exercise 2 · Check a join (6 min)

Answer: 32,510 rows before, 32,510 after, every row `both` .

Make them write the prediction down before running it. The prediction is the exercise, the merge is just the check. A student who cannot predict the row count does not yet know what their rows are.

Things to watch:

Merging the other way round, `tracks.merge(membership, ...)` . It runs, and it quietly changes what a row means. Ask what one row is now.

`how="inner"` . Also runs, also gives 32,510 rows here, and would silently drop unmatched memberships on any less tidy snapshot. The left join states the intent.

Dropping `validate=` because it "did not do anything". It did: it asserted the assumption. Answer the two-rows-on-the-right question concretely. One membership becomes two output rows, and every count and mean built on it is inflated. The duplicate-rows meme in the student notebook exists for this moment.

```
# Solution 2: check cardinality and matching keys.
practice_join = membership.merge(
    tracks, on="track_id", how="left", validate="many_to_one", indicator=True
)
print("Membership rows:", len(membership))
print("Joined rows:", len(practice_join))
assert len(practice_join) == len(membership)
display(practice_join["_merge"].value_counts())
```

```
Membership rows: 32510
Joined rows: 32510
_merge
both          32510
left_only      0
right_only     0
Name: count, dtype: int64
```

Exercise 3 · Conditional grouping (8 min)

At `energy >= 0.8` : edm 3,433, rock 2,232, pop 1,774, latin 1,586, rap 1,146, r&b 717.

At `0.7` : edm 4,659, **pop 3,115, rock 3,032**, latin 2,935, rap 2,394, r&b 1,591.

Moving the threshold reorders the table. Rock is behind pop at 0.7 and ahead of it at 0.8, and across the demo cell below it climbs from 5th at 0.6 to 2nd at 0.8 while edm and r&b never move. Best thirty seconds in the notebook, so run it live. A working definition is not a neutral choice, and the ranking it produces belongs as much to the cutoff as to the music.

Things to watch:

Filtering after grouping rather than before, or filtering `df` instead of `joined` .

Reading counts as popularity. edm leads partly because edm has more associations to start with. Push them to the share, `count / association_count` from `summary` .

Causal language, "edm makes people energetic". These are playlist placements labelled by playlist genre, in one 2020 snapshot.

```
# Solution 3, step 1 – filter first, then group the filtered rows.
high_energy = joined.loc[joined["energy"] >= 0.8]
high_energy_summary = (high_energy.groupby("genre", dropna=False)
                        .agg(high_energy_associations=("track_id", "size"),
                             mean_danceability=("danceability", "mean"))
                        .sort_values("high_energy_associations", ascending=False))
display(high_energy_summary.round(3))
```

	high_energy_associations	mean_danceability
genre		
edm	3433	0.643
rock	2232	0.489
pop	1774	0.627
latin	1586	0.716
rap	1146	0.664
r&b	717	0.679

```
# Solution 3, step 2 – the modification the exercise asks for: lower the threshold
and compare counts.
moderately_high_energy = joined.loc[joined["energy"] >= 0.7]
threshold_07_summary = (moderately_high_energy.groupby("genre", dropna=False)
                        .agg(energy_07_associations=("track_id", "size"))
                        .sort_values("energy_07_associations", ascending=False))
display(threshold_07_summary)
```

energy_07_associations	
genre	
edm	4659
pop	3115
rock	3032
latin	2935
rap	2394
r&b	1591

```
# Live demo: the ranking is a property of the threshold.
ranks = {}
for threshold in (0.6, 0.7, 0.8, 0.9):
    counts = joined.loc[joined["energy"] >= threshold].groupby("genre").size()
    ranks[f">= {threshold}"] = counts.rank(ascending=False).astype(int)
display(pd.DataFrame(ranks).sort_values(">= 0.8"))
```

	>= 0.6	>= 0.7	>= 0.8	>= 0.9
genre				
edm	1	1	1	1
rock	5	3	2	2
pop	2	2	3	3
latin	3	4	4	4
rap	4	5	5	5
r&b	6	6	6	6

Exercise 4 · Reshape and interpret (7 min)

Answer: one row of `genre_feature_long` is one genre-measure mean, say the mean danceability of pop associations.

By mean danceability: rap 0.718, latin 0.714, r&b 0.670, edm 0.655, pop 0.639, rock 0.520. By mean energy: edm 0.801, rock 0.733, latin 0.711, pop 0.701, rap 0.651, r&b 0.591. Rock is second on energy and last on danceability, a useful reminder that these are different constructs and not two ways of saying "lively".

Things to watch:

Melting the count columns too, producing rows where `measure` is `association_count`. Technically long, semantically nonsense. Means and counts are not the same kind of quantity.

"Long format lost information." Nothing was lost. The same numbers are addressed by `(genre, measure)` instead of by column name, and `pivot_table` goes back.

Comparing across measures. A 0.1 difference in energy is not a 0.1 difference in danceability. Both are bounded 0 to 1, which is why they share an axis, and that is a display convenience rather than a claim of equivalence.

A caption with no limitation in it. The exercise asks for one result and one snapshot limitation. Accept nothing without both.

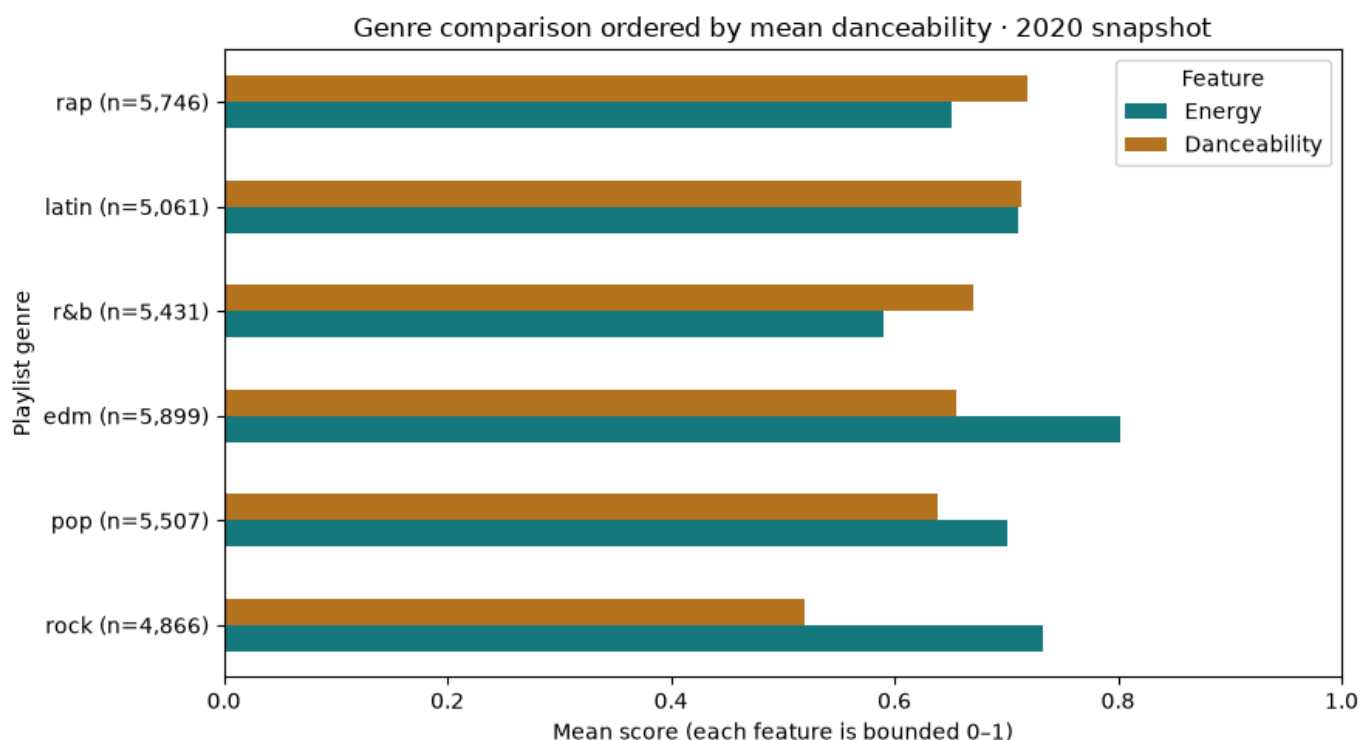
```
# Solution 4, step 1 – read the answer off the long table.
danceability_long = genre_feature_long.loc[genre_feature_long["measure"] ==
"danceability"]
display(danceability_long.sort_values("mean_value", ascending=False))
```

	genre	measure	mean_value
10	rap	danceability	0.718353
8	latin	danceability	0.713884
11	r&b	danceability	0.670179
6	edm	danceability	0.654714
9	pop	danceability	0.639302
7	rock	danceability	0.520037

```
# Solution 4, step 2 – reorder the rows and rebuild the labels for the new order.
plot_data_by_danceability = summary.sort_values("mean_danceability")
danceability_labels = [f"{genre} (n={count:},)"
                       for genre, count in
plot_data_by_danceability["association_count"].items()]
print(danceability_labels)
```

```
['rock (n=4,866)', 'pop (n=5,507)', 'edm (n=5,899)', 'r&b (n=5,431)', 'latin (n=5,061)', 'rap (n=5,746)']
```

```
# Solution 4, step 3 – the same chart, ordered by danceability. The wide summary
agrees with the long table.
ax = plot_data_by_danceability[["mean_energy", "mean_danceability"]].plot(
    kind="barh", figsize=(9, 5), color=["#187c80", "#b87322"]
)
ax.set(title="Genre comparison ordered by mean danceability · 2020 snapshot",
        xlabel="Mean score (each feature is bounded 0–1)",
        ylabel="Playlist genre", xlim=(0, 1))
ax.set_yticks(np.arange(len(plot_data_by_danceability)))
ax.set_yticklabels(danceability_labels)
ax.legend(["Energy", "Danceability"], title="Feature")
plt.tight_layout()
plt.show()
```



Optional extension · Sampling and correlation (after class)

Not a learning gate, and not worth displacing the merge or the reshape. If someone asks in class, the worked version is below.

The correlations are small and their signs differ by genre: about -0.15 in edm, rock and rap, $+0.12$ to $+0.16$ in latin and r&b, -0.03 in pop. So there is no single energy-danceability relationship here to report.

A 2,000-row sample leaves roughly 330 rows per genre and shifts the estimates by up to 0.11. edm's weak negative association all but disappears, even though the signs happen to survive. Magnitudes off a few hundred rows are not something to quote.

```

# Worked version of the optional starter.
sample = joined.sample(n=min(2_000, len(joined)), random_state=2026)

full_corr = joined.groupby("genre")["energy"].corr(joined["danceability"])
sample_corr = sample.groupby("genre")["energy"].corr(sample["danceability"])

comparison = pd.DataFrame({
    "full_snapshot": full_corr,
    "sample_2000": sample_corr,
    "rows_in_sample": sample.groupby("genre").size(),
})
display(comparison.round(3))

```

	full_snapshot	sample_2000	rows_in_sample
genre			
edm	-0.150	-0.041	348
latin	0.115	0.229	323
pop	-0.031	-0.017	340
r&b	0.159	0.211	329
rap	-0.148	-0.154	341
rock	-0.151	-0.106	319

The appendix cells in the student notebook

The student notebook ends with two appendix cells: the regular-expression version of the date-precision check, and the grouped- `nunique` version of the track-consistency check. Both finish by asserting they reach the same conclusion as the short versions in the main path.

They are labelled as later material. If someone runs ahead and asks, the honest framing is that the short version is exact because the class snapshot is frozen and clean, and the appendix version is what you write when you do not control the file. Do not teach regular expressions today.

Exit-ticket answers you are listening for

List vs `ndarray` vs `DataFrame` . A list holds any objects and needs a loop to compute. An `ndarray` holds one dtype in a shape and computes on everything at once. A `DataFrame` adds labelled columns of possibly different dtypes plus an index, so rows and columns can be picked by name. Accept anything that names loop, then vectorised, then labelled. A student who only says "a `DataFrame` is a table" has not got it yet.

One limitation. Any of these: a row is a track-playlist-genre association, so a popular song counts more than once; the left join assumes `track_id` is unique in the track table and validation is what enforces that; `2018` is a real release date at year precision and parsing it to 1 January invents a day; `price_times_enrollments_proxy` is a constructed proxy and not observed revenue; the Spotify data is a 2020 snapshot and says nothing about now.

Weak answer worth pushing on: "the data might be biased." Which row? Which assumption?